Real-time movie analytics on Google Cloud

Scientific report

BARONCEA Andrei-Florin and KERESTELY Alexandra-Nicola

April 27, 2026

Abstract

This is the scientific report for the real-time movie analytics pipeline deployed on Google Cloud Platform. It covers the system architecture, how the services communicate, a comparison to Netflix Keystone, and a conclusion. The consistency, performance, and resilience sections include results from the measurement runs available in the repository.

Contents

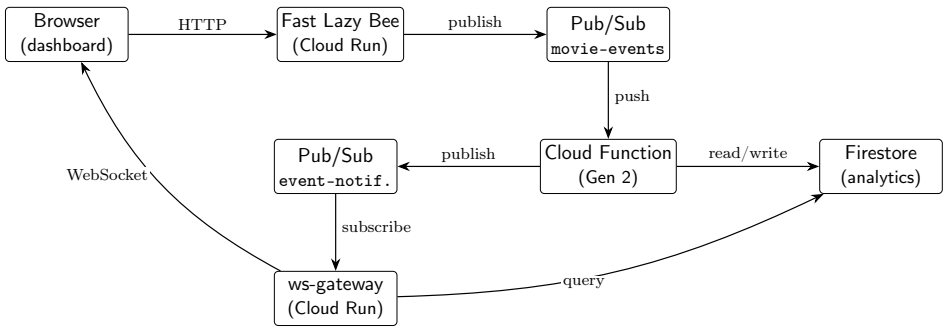
1 Architecture

This coursework extends the Fast Lazy Bee REST API with an event-driven analytics module deployed on Google Cloud Platform. The sample movie catalogue remains in MongoDB (accessed only by Fast Lazy Bee). The analytics module and the WebSocket gateway Firestore collections (`movie-stats`, `processed-messages`) with idempotent processing on the serverless consumer. The WebSocket gateway periodically refreshes movie statistics aggregates from Firestore and merges them into snapshot messages together with recent actions derived from `event-notifications` which are broadcasted to clients (`dashboard`). Pub/Sub topics `movie-events` (ingested by the analytics function) and `event-notifications` (downstream fan-out) connect the pipeline.

Deployment of the components:

- Fast Lazy Bee - Cloud Run
- MongoDB movie database - Atlas Cluster
- Analytics function - Cloud Run
- WebSocket gateway - Cloud Run
- Dashboard client - local

Figure 1 summarises the logical data flow from a successful movie read to pushed dashboard statistics. Boxes denote deployable or managed services; arrow labels name the interaction style (protocol or product API).



2 Communication

Service Fast Lazy Bee exposes a conventional synchronous REST communication: the browser issues `GET /movies/:id` over HTTPS and waits for the JSON response. Publishing to Pub/Sub is triggered after the read succeeds.Everything downstream of `movie-events` is **asynchronous**: Pub/Sub

may batch or retry, the Cloud Function runs when messages arrive, and the WebSocket gateway consumes `event-notifications` and broadcasts them to the clients through the WebSocket protocol.

Table 1 lists the main communication links.

Link	Pattern	Rationale	Technology
Browser → Fast Lazy Bee	Sync	Standard REST CRUD; user waits for HTTP 200 and body.	HTTPS; Fastify on Cloud Run (`fast-lazy-bee`).
Fast Lazy Bee → Pub/Sub	Async	Decouple API latency from analytics; absorb spikes without blocking the response path beyond local publish latency.	Pub/Sub topic `movie-events`.
Pub/Sub → Analytics Function	Async	Managed push delivery invokes the function with CloudEvents.	Cloud Functions (2nd gen); subscriber on `movie-events`.
Analytics Function → Analytics Database	Sync	Deterministic read-before-write and idempotency checks inside one invocation.	Firestore (`movie-stats`, `processed-messages`).
Function → notification path → WebSocket gateway	Async	Function publishes a compact notification; gateway's subscription delivers it without coupling function lifetime to WebSocket fan-out.	Pub/Sub topic `event-notifications`; gateway pull subscriber.
Browser ↔ WebSocket gateway	Async push (duplex channel)	Server pushes `stats.snapshot` updates without client polling (broadcast); connection stays open.	WSS to Cloud Run (`ws-gateway`); token query auth.

Table 1: Synchronous vs asynchronous interactions

3 Consistency and CAP

3.1 Operational data plane (MongoDB / Service A)

The Fastify service reads movie documents from **MongoDB** (Atlas, in the same cloud region as the GCP workloads). Reads follow standard *single-document* consistency: a `GET /movies/:id` returns one coherent document for that key.

3.2 Analytics plane (Pub/Sub, Cloud Function, Firestore)

View events are published to **Pub/Sub** and processed by the **analytics function**, which writes to `movie-stats` documents in **Firestore** and republishes to `event-notifications` for the WebSocket gateway. This path is inherently **asynchronous**. The function uses **idempotency** checks on the incoming Pub/Sub `messageId` to avoid counting duplicate deliveries. As a result, the data shown on the dashboard is **eventually consistent**: a new view only appears after Pub/Sub delivery, function execution, a Firestore commit, and the next gateway snapshot or timer tick.

3.3 User-visible delay and measured windows

End-to-end delay from an HTTP view until a `stats.snapshot` on the WebSocket reflects that movie is measured by: five warmup iterations are discarded, then $n = 35$ samples are recorded. Separately, another script publishes synthetic `movie_viewed` events and polls Firestore until at least one `movie-stats` row appears for a new `movieId`; Table 2 shows results from one representative CSV export.

Table 2: Measured analytics visibility. Convergence: time until Firestore shows the first stat for a new id ($n = 5$ runs). Burst: after 100 concurrent publishes to the same id, Firestore `view_count` reached the target within the sampled timeline.

Metric	Value
Mean convergence delay (Pub/Sub → Firestore visible)	1.11 s
Final `movie-stats` count after burst ($t = 20$ s)	100

The burst phase shows some catch-up lag under load: at $t = 15$ s only 72 of 100 publishes had completed and Firestore held 64 rows for that movie id, which is consistent with the script's console output (publish futures vs. Firestore row count).

3.4 CAP brief

The system runs in **one region**. **Partition tolerance** is assumed in a limited sense: Pub/Sub and Firestore stay available under normal cloud operation, but the design does not claim global consistency across MongoDB, Pub/Sub, and Firestore. If a regional fault occurs, the dashboard path may become unavailable (gateway or function down), while the **consistency** of what the user sees is limited by the asynchronous pipeline.

3.5 Client behaviour

The static dashboard (`dashboard/app.js`) reconnects with **exponential backoff and jitter** (capped at 60 s). When a new WebSocket connection opens, the gateway sends a welcome message followed by `stats.snapshot` updates; each snapshot replaces the displayed popular and recent lists through `applySnapshot`. After a disconnect, a successful reconnect gives the user a **fresh** stream of `stats.snapshot` messages without needing to reload the page.

4 Performance

End-to-end latency. The test script sends a `GET` on `/movies/{id}` while a WebSocket connection to the gateway is open, then waits for the next `stats.snapshot` whose timestamp has advanced and that includes the same movie id. Latency is measured as wall-clock time from when the HTTP response body finishes reading until the snapshot arrives. **Warm-up:** the first five iterations are excluded from the statistics; **sample size:** $n = 35$ counted iterations.

Table 3: End-to-end view-to-snapshot latency (ms, one decimal).

Statistic	Value
p_{50}	8849.1
p_{95}	9654.7
min / max	254.5 / 10121.4

HTTP throughput under staged concurrency. The test script runs three back-to-back stages of **20 s** each with worker counts of $(1, 3, 5)$: within each stage it repeatedly sends batches of that many parallel `GET` requests until the time window ends. The script sets `STAGES = (1, 3, 5)` and `STAGE_SECONDS = 20`. Figure 1 shows the sustained throughput per stage, calculated as the number of attempts divided by the observed wall time for that stage in the CSV.

Table 4: HTTP load test summary. Source: `load-http-20260427-100324.csv`. All listed attempts returned HTTP 200 (0 errors in this run).

Stage	Workers	Attempts	Throughput (req/s)
0	1	77	3.9
1	3	195	9.9
2	5	240	12.0

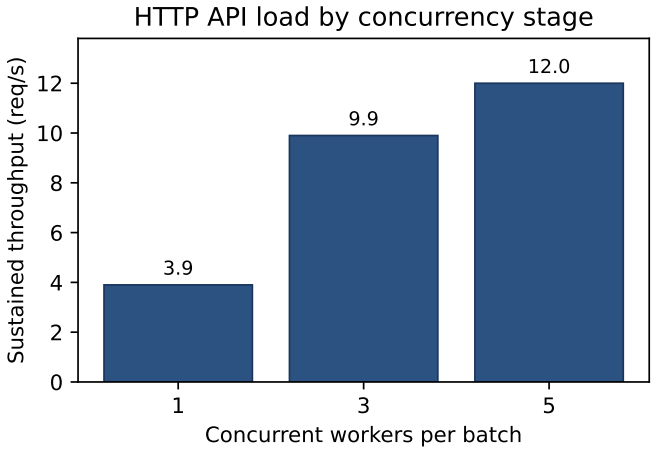


Figure 1: Sustained HTTP request rate vs. worker concurrency. Each stage used a 20 s window; concurrency increased from 1 to 3 to 5 workers per batch.

5 Resilience

5.1 Evidence from load and error metrics

The staged HTTP load test recorded attempts across increasing concurrency with **no non-200 HTTP responses**, giving an observed error rate of 0 for that run. This shows that the Service A read path stayed available under moderate parallel load.

5.2 Client reconnect and fresh state

The dashboard client reconnects using exponential backoff with multiplicative jitter, capped at 60 s between attempts. After each successful reconnect, the gateway sends a welcome message and resumes `stats.snapshot` payloads; the client applies each snapshot to the DOM, so the user sees an up-to-date `popularMovies` / `recentActions` view without refreshing the page.

6 Comparison to a real system

This section compares the project pipeline to a **real** large-scale deployment: Netflix's Keystone real-time stream processing platform, described in Netflix's engineering blog [1]. The goal is not to claim similar throughput, but to show that the *same architectural patterns* appear in both: append-oriented events, managed messaging, stream-style processing, and fan-out to consumers.

6.1 Pattern mapping

Keystone ingests microservice events at very high volume, routes them through Kafka-style messaging layers, and supports both batch and stream processing as a managed platform. Our system is much smaller but follows the same structure: Service A emits a movie_viewed event after a successful read; Google Cloud Pub/Sub provides durable, asynchronous messaging between producers and subscribers [2]; a serverless function writes idempotently before sending a downstream notification; the WebSocket gateway pushes snapshots to browsers. Industry documentation for queue-based systems emphasises at-least-once delivery and therefore idempotent consumers [3], which is exactly what the processed-messages deduplication design in this repository provides.

Table 5 summarises the mapping. "Shared pattern" highlights what is conceptually the same between the two systems; "Difference" notes scale and product choices without claiming numerical equivalence.

This project (GCP)	**Keystone (Netflix)**	**Shared pattern**	**Difference (scale / product)**
`movie-events` Pub/Sub topic	Fronting Kafka clusters	Log-oriented ingest buffer; decouple producers from processors	Regional Pub/Sub vs. global Kafka estate; message volume
Cloud Function (Gen 2) consumer	Stream processing / routing services	Asynchronous data processing	Single-function demo vs. multi-tenant SaaS
Firestore `movie-stats`	Derived stores / OLAP paths in Netflix stack	Recorded analytics from events	Document store vs. warehouse / specialised stores
`event-notifications` + gateway subscription	Internal routing toward real-time apps	Pub/Sub communication isolates writers from fan-out	Pub/Sub pull to Cloud Run vs. Netflix-internal fabric
WebSocket `stats.snapshot`	Near-line dashboards and operational views	Push updates without polling	Minimal fan-out vs. global UI traffic

Table 5: Architectural comparison: coursework pipeline vs. Netflix Keystone

6.2 Takeaway

Netflix describes Keystone as a unified event publishing, collection, and routing layer for both batch and stream processing [1]. This project applies the same decouple–process–notify approach using native GCP primitives [2], with explicit handling of redelivery semantics. The main difference is

scale and operational depth: systems like Netflix add partitioning strategies, multi-region durability, and large-scale observability that are out of scope here, but the underlying logical structure is the same.

7 Conclusion

Goal. This project extends a REST movie API into an event-driven analytics pipeline on Google Cloud, with live dashboard updates matching the requirement for decoupled services, managed messaging, a FaaS consumer, and a WebSocket channel.

What was built. The system consists of independently deployable components: a Cloud Run API, Pub/Sub topics `movie-events` and `event-notifications`, a Cloud Function, Firestore-backed aggregates with idempotent processing, a Python WebSocket gateway, and a static dashboard. Together they ensure that a successful `GET /movies/:id` can eventually appear as a pushed `stats.snapshot` to connected clients, as shown in Section 1 and Table 1.

Evidence and lessons. The architecture and synchronous versus asynchronous communication are described above. Latency percentiles, throughput results, consistency windows, and fault tests are covered in the Performance and Resilience sections. The main lessons from this project align with standard industry practice: queue-backed decoupling, idempotent consumers under at-least-once delivery, and a separate notification leg so that write paths are not blocked by fan-out. These same patterns appear in the Netflix Keystone comparison (Section 6) and in cloud messaging documentation [2].

8 AI Usage

Cursor IDE and ChatGPT were used during development.

Bibliography

[1] Netflix Technology Blog. *Keystone: Real-time Stream Processing Platform*. Jan. 2018. URL: https://netflixtechblog.com/keystone-real-time-stream-processing-platform-a3ee651812a (visited on 04/26/2026).

[2] Google Cloud. *Pub/Sub documentation: Reliable, many-to-many, asynchronous messaging*. 2026. URL: https://cloud.google.com/pubsub/docs/overview (visited on 04/26/2026).

[3] Kafka Confluent Documentation. *Kafka Message Delivery Guarantees*. 2026. URL: https://docs.confluent.io/kafka/design/delivery-semantics.html (visited on 04/26/2026).